Kalibrier-Checkliste Positionsbestimmung

Odometrie (Räder) + Radar-Fusion · `systemlib.cpp` · Teensy 4.0

Reihenfolge einhalten: Spätere Schritte bauen auf früheren auf. Alle `FUS_*` und `ODO_*`-Werte stehen am Anfang des Fusionsblocks in `systemlib.cpp`, die `POS_*`-Werte in `params.h`. Nach jeder Änderung neu flashen und den Test wiederholen.

Debug-Kanäle (PDS)

Kanal	Inhalt
20–23	Drehzahl VL / VR / HL / HR (`Speed_real`)
190 / 191	Feldgeschwindigkeit der Odometrie vx / vy [cm/s]
192 / 193	Innovation X / Y = Radar minus Vorhersage [cm]
194 / 195	Unsicherheit σ X / Y [cm]
196 / 197	Schätzung, um wie viele ms `FUS_RADAR_LATENCY_MS` falsch liegt (positiv = erhöhen)

0 Geometrie und Radar-Basis

Alles andere hängt davon ab. Feldmaße und Radar-Offset zuerst.

Wert	Default	Zweck	Neu / gemessen
`POS_FIELD_X_CM / _Y_CM`	params.h	Feldmaß (Breite / Länge)	
`POS_RADAR_OFFSET_CM`	params.h	Abstand Sensor → Roboter-Mitte	
`WHEEL_RADIUS_CM`	params.h	Radradius (Umrechnung rpm → cm/s)	
`GEAR_RATIO`	params.h	Getriebeübersetzung	
`POS_PAIR_TOL_CM`	params.h	erlaubte Abweichung Li+Re gegen Feldbreite	

- ☐ **Dist in mm:** Heading 0, Roboter vor eine Wand, Abstand mit Maßband messen. `Dist/10` muss dem Maßband entsprechen (der Header sagt „Counts“, der Code rechnet als mm).
- ☐ **Radar-Offset:** Roboter mit Heading 0 an beliebige X-Position. Offset = (Feldbreite − Li − Re) / 2, Li und Re in cm. Für Y mit Vo und Hi, dafür seitlich vor das Tor stellen. An 5 Positionen wiederholen, Streuung < 0,5 cm.
- ☐ **Übersetzung / Radius:** Roboter anheben, ein Rad ~10 s mit fester Drehzahl laufen lassen, Radumdrehungen zählen (Markierung). Muss `Speed_real / GEAR_RATIO` entsprechen. Radius mit Messschieber bis zur Rollen-Außenkante.
- ☐ **Paar-Toleranz:** `|Li + Re + 2·Offset − Feldbreite|` im Stand und in Fahrt loggen. Toleranz mindestens das Dreifache des größten normalen Werts.

1 Odometrie (nur Räder)

Temporär für diesen Abschnitt: `FUS_LOST_SIGMA = 1000`, `FUS_P_INIT = 1` und `calculatePosition(..., use_radar=false)`. Sonst gibt die Position nach wenigen Sekunden −1 aus, weil ohne Radar die Unsicherheit wächst. Danach zurücksetzen (40 bzw. 900).

Wert	Default	Zweck	Neu / gemessen
`ODO_SCALE_X`	1.000	Skalierung quer	
`ODO_SCALE_Y`	0.845	Skalierung vor/zurück	
`FUS_V_DEADBAND`	2.0 cm/s	darunter gilt der Roboter als stehend	

- ☐ **Skalierung:** Bei Heading 0 exakt 100 cm längs fahren (Maßband). Neuer Faktor = alter × gemessen / angezeigt. 5× je Richtung, bei ~30 % und ~80 % Tempo; danach quer. Weicht das Ergebnis je nach Tempo ab, liegt es am ADC-Mapping, nicht am Faktor.
- ☐ **Diagonaltest:** 100 cm im 45°-Winkel. Beide Achsen müssen ~70,7 cm zeigen.
- ☐ **Heading-Test:** Auf 90° drehen, 100 cm fahren. Richtung und Betrag prüfen.

- ☐ **Deadband:** 60 s stehen lassen, Kanal 190/191 beobachten. Wert knapp über das maximale Rauschen setzen. Die Position darf in 60 s um < 1 cm driften.
- ☐ **Zurücksetzen:** `FUS_LOST_SIGMA = 40`, `FUS_P_INIT = 900`, Radar wieder an.

2 Zeitwerte (gegen das Hinterherhinken)

Wert	Default	Zweck	Neu / gemessen
`FUS_RADAR_LATENCY_MS`	30	Alter des Radarwerts	
`FUS_MOTOR_LAG_MS`	20	Verzögerung der Motor-Rückmeldung	
`FUS_MAX_ACCEL`	600 cm/s²	Grenze für die Extrapolation	
`FUS_ACCEL_LPF`	0.35	Glättung der Beschleunigung	
`FUS_HIST_N`	128	Länge der Positionshistorie	

- ☐ **Radar-Latenz:** Ohne Beschleunigung mit ~50 cm/s hin- und herfahren. Kanal 196/197 muss im Mittel nahe 0 liegen (Richtwert ±5 ms). Sonst den angezeigten Wert zur Konstanten addieren.
- ☐ **Motor-Lag:** Hart anfahren und bremsen, Innovation (192/193) soll um 0 pendeln. Beim Anfahren in Fahrtrichtung positiv und beim Bremsen negativ → Lag erhöhen; umgekehrtes Vorzeichen → senken.
- ☐ **Max. Beschleunigung:** Vollgas aus dem Stand, Steigung von Kanal 190/191 messen. Der Wert muss darüber liegen.
- ☐ **Glättung:** Zittert die Position beim Anfahren → `FUS_ACCEL_LPF` senken. Bleibt Rest-Hinterherhinken → erhöhen.
- ☐ **Historie:** Es muss gelten `FUS_HIST_N × Zykluszeit > 2 × Radar-Latenz`. Zykluszeit: `giveCycleTime_us()`.

3 Filter-Vertrauen

Wert	Default	Zweck	Neu / gemessen
`FUS_SIGMA_RADAR_CM`	2.5	Vertrauen ins Radar (größer = mehr Räder)	
`FUS_Q_BASE`	1.0 cm²/s	Grunddrift	
`FUS_Q_SPEED`	0.15	relativer Radfehler	
`FUS_Q_ROT`	1.0	Schlupf beim Drehen	
`FUS_GATE_SIGMA`	3.0	Ausreißer-Gate	

- ☐ **Radar-Vertrauen:** Im Stand die Standardabweichung des Radar-Rohwerts messen, Wert auf das 3–5-Fache setzen. In Fahrt soll die Innovation um 0 pendeln und selten über ±2–3 cm gehen.
- ☐ **Q_SPEED:** Schnell fahren. σ (Kanal 194/195) sollte bei ~1–3 cm bleiben und die Innovation selten das Gate überschreiten. σ deutlich größer → Q zu hoch; Gate reißt oft → Q zu klein.
- ☐ **Q_ROT:** 360° auf der Stelle mit 1–2 U/s drehen. Positionsfehler danach (Markierung am Boden) innerhalb von 1–2σ.
- ☐ **Gate:** Hand 2 s vor den Li-Sensor halten. Die Position darf sich höchstens 1 cm bewegen.

4 Hindernis, Sprung, Neusetzen

Wert	Default	Zweck	Neu / gemessen
`FUS_REACQ_MS`	300	stabil abweichender Wert → Position neu setzen	
`FUS_REACQ_OBST_MS`	1500	dasselbe, wenn es wie ein Hindernis aussieht	
`FUS_REACQ_TOL_CM`	4.0	„stabil“ = Werte streuen weniger als das	

- ☐ **Anschieben:** Roboter hochnehmen, 30 cm daneben absetzen. Die Position muss nach etwa `FUS_REACQ_MS` stimmen.

☐ **Hindernis:** Gegner vor den Li-Sensor halten. Die Position darf nicht springen. Nach `FUS_REACQ_OBST_MS` übernimmt sie den falschen Wert (gewollt). Steht in deinen Spielen länger ein Gegner davor → erhöhen.

5 Radar-Nutzung und Heading

Wert	Default	Zweck	Neu / gemessen
`FUS_MAX_TILT_DEG`	25°	Radar nur nutzen, wenn Heading so nah an 0° / 180°	
`calibBNO()`	–	Heading-Nullpunkt	

☐ **Winkelgrenze:** Roboter an einen markierten Punkt stellen, schrittweise um 10°, 20°, 30°, 40° drehen, Innovation loggen. Grenze dorthin setzen, wo der Fehler über ~2 cm steigt.

☐ **180°-Umschaltung:** Auf 180° drehen. Die Innovation muss klein bleiben, sonst ist die Sensorzuordnung falsch.

☐ **BNO-Drift:** 5 min stillstehen, Drift < 1°. Nach voller Drehung Rückkehr auf 0 ± 1°. 2° Winkelfehler ≈ 3,5 cm Positionsfehler bei 100 cm Fahrt.

6 Gültigkeit und Verlust

Wert	Default	Zweck	Neu / gemessen
`FUS_VALID_SIGMA`	5 cm	darunter XValid / YValid	
`FUS_LOST_SIGMA`	40 cm	darüber Position = −1	
`FUS_P_INIT`	900 cm²	Startunsicherheit	
`FUS_ASSUMED_SPEED`	60 cm/s	Q ohne Odometrie	
`FUS_MIN_CERT`	0.05	Untergrenze Radar-Certainty	

☐ **Radar-Ausfall:** Alle Radar-Sensoren abdecken und fahren. Zeit bis `XValid` false wird und bis −1 kommt messen. Muss zur Spiellogik passen.

☐ **Start:** Roboter an die Startposition setzen. Die Position muss innerhalb von ~1 s auf σ < 5 cm konvergieren.

7 Bestehende Plausibilitätswerte (nur prüfen)

Diese Werte funktionieren bereits, hier nur kontrollieren.

`POS_BLIND_CM`, `POS_MAX_OVERSHOOT_CM`, `POS_PLAUS_*`, `POS_AGE_MAX_MS`, `POS_GOAL_WIDTH/DEPTH/EDGE_CM` sowie die Radar-Abstimmung in `params.h`.

☐ **Tor-Maße** mit Maßband prüfen. Danach ins Tor fahren: Die Innovation in Y muss dort klein bleiben.

✓ Abnahmetest zum Schluss

☐ Rechteck 5× mit voller Geschwindigkeit fahren und genau an der Startmarkierung stoppen. Endfehler < 3 cm.

☐ Roboter einmal wegschieben: Er muss sich innerhalb von ~0,5 s wieder fangen.

☐ Innovation (192/193) pendelt in Fahrt um 0, σ (194/195) bleibt bei ~1–3 cm.